HTTP & Web APIs (requests & httr2)

Lecture 14

Dr. Colin Rundel

Reading directly from the web

Most functions that read from a file path will also accept a URL, downloading the file for you behind the scenes.

```py
import pandas as pd
import urllib.request

url = "https://sta523-fa26.github.io/slides/dat
```

```py
pd.read_csv(url).shape
```

```
(344, 8)
```

```py
urllib.request.urlopen(url).readline()
```

```
b'species,island,bill_length_mm,bill_depth_mm,fl
```

```py
urllib.request.urlretrieve(url, "penguins.csv")
```

```r
url = "https://sta523-fa26.github.io/slides/dat
```

```r
dim(read.csv(url))
```

```
[1] 344   8
```

```r
dim(readr::read_csv(url, show_col_types = FALSE
```

```
[1] 344   8
```

```r
readLines(url, n = 1)
```

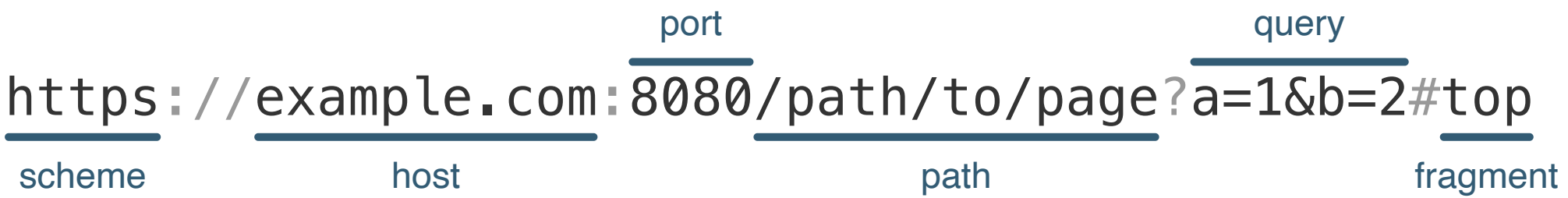
```
[1] "species,island,bill_length_mm,bill_depth_mm
```

```r
download.file(url, "penguins.csv")
```

This is convenient for a public file at a fixed URL, but there is no way to set headers, authenticate, pass parameters, or handle a failed request. For that we need to work with HTTP directly.

HTTP

URLs



Component	Description
scheme	The protocol to use, usually `http` or `https`
host	The domain name or IP address of the server
port	The port to use, defaults to 80 for `http`, 443 for `https`
path	Identifies the resource on the server
query	Optional key-value pairs
fragment	A location within the page, not sent to the server

URL encoding

Only a limited set of characters may appear in a URL, so spaces and reserved characters like `&`, `/`, and `?` are *percent encoded*. These values are usually not part of the host or path but can occur as part of the query string.

```py
from urllib.parse import urlencode, quote, unquote
```

```py
urlencode({"q": "hello world", "n": 10})
```

```
'q=hello+world&n=10'
```

```R
URLencode("hello world")
```

```
[1] "hello%20world"
```

```py
quote("a/b?c=d&e", safe="")
```

```
'a%2Fb%3Fc%3Dd%26e'
```

```R
URLencode("a/b?c=d&e", reserved = TRUE)
```

```
[1] "a%2Fb%3Fc%3Dd%26e"
```

```py
unquote("hello%20world")
```

```
'hello world'
```

```R
URLdecode("hello%20world")
```

```
[1] "hello world"
```

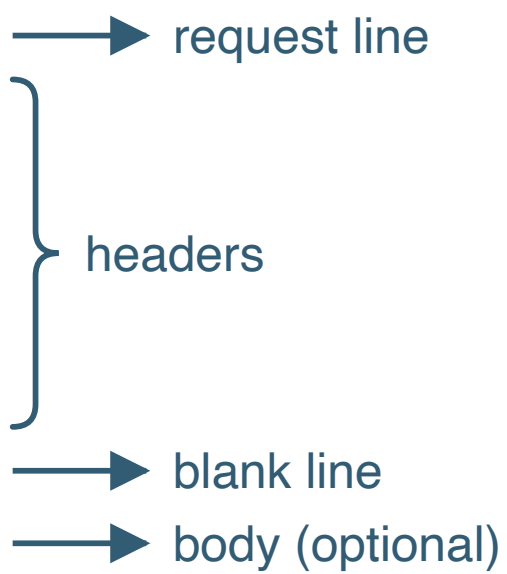
Implementations can be inconsistent (e.g. spaces can be encoded as `+` or `%20`) but consumers are generally permissive. Both `requests` and `httr2` encode parameter values automatically.

HTTP requests

An HTTP request is made up of a method, a URL, headers, and an optional body.

```
GET /path/to/page?a=1&b=2 HTTP/1.1
Host: example.com:8080
User-Agent: httr2/1.2.1
Accept: application/json
Accept-Encoding: gzip, deflate
Connection: keep-alive

(empty for a GET, JSON for a POST)
```



HTTP methods

The method, or verb, says what we want done with the resource at the URL.

Method	Purpose	GitHub API example
`GET`	Fetch a resource	Get a user, list org repos
`POST`	Create a new resource	Create a gist, open an issue
`PUT`	Replace a resource	Replace a file in a repo
`PATCH`	Partially update a resource	Update a repo's description
`DELETE`	Remove a resource	Delete a gist

`GET` and `POST` are by far the most common when consuming APIs.

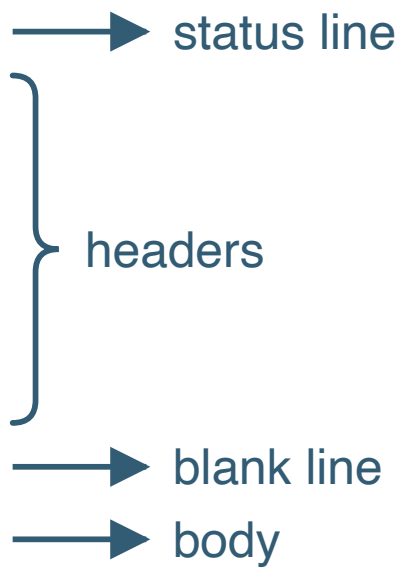
Loading a page in a browser is a `GET`, and submitting a form is usually a `POST`.

HTTP responses

An HTTP response contains a status, headers, and an optional body (HTML, JSON, an image, or other content).

```
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2026 15:30:00 GMT
Server: nginx/1.27.2
Content-Type: text/html; charset=utf-8
Content-Length: 20
Connection: keep-alive

<h1>Hello world</h1>
```



Status codes

The first digit gives the category. `1xx` is informational, `2xx` is success, `3xx` concerns redirection or caching, `4xx` a problem with the request, and `5xx` a problem on the server.

Code	Description	Meaning
200	OK	Request succeeded
201	Created	Resource created, usually after a `POST`
204	No Content	Success with no response body
301	Moved Permanently	Permanent redirect
400	Bad Request	Malformed request or parameters
401	Unauthorized	Authentication required or failed
403	Forbidden	Server refuses access, may also mean rate limited
404	Not Found	Resource not found or not visible to this client
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server-side failure
503	Service Unavailable	Server temporarily unavailable

The code is only a summary. For an error, the response body will usually include a more specific message explaining what went wrong, so it is worth reading.

`requests`

`requests` is the standard third-party Python package for HTTP. A call such as `requests.get()` builds and sends a request in one step and returns a response object. Its attributes and methods give the status, headers, and parsed body.

```py
requests.get(
  url, params=..., headers=..., timeout=10
)
requests.post(url, json=...)
```



httr2

`httr2` separates building a request from performing it. A request object is built up with `req_*()` functions, sent with `req_perform()`, and the resulting response is inspected with `resp_*()` functions.

```r
request(url) |>
  req_url_query(...) |>
  req_headers(...) |>
  req_timeout(10) |>
  req_perform()
```



REST APIs

REST

Representational State Transfer is an architectural style for web services, not a standard. A RESTful API exposes resources at URLs, called endpoints, and clients work with them using ordinary HTTP.

- Resources are addressable. Every resource has its own URL, such as `/users/rundel` or `/orgs/tidyverse/repos`.
- Requests are stateless. Each request carries everything the server needs to handle it, so nothing is remembered between requests.
- The interface is uniform. The standard HTTP methods say what to do and the status codes say how it went.
- Resources are returned as representations in the response body. JSON is the most common format, but REST does not require it.

Last lecture we scraped data out of HTML meant for people. APIs provide structured data meant for programs.

GitHub API

GitHub's REST API exposes nearly everything on the site. We will use it as a running example because it is well documented, mostly public, and the responses are plain JSON.

GitHub REST API

Endpoints used today,

- Get a user, `GET /users/{username}`
- List organization repositories, `GET /orgs/{org}/repos`
- Get the authenticated user, `GET /user`
- Create a gist, `POST /gists`

A first request

```py
api = "https://api.github.com"
resp = requests.get(
  api + "/users/rundel", timeout=10
)
resp
```

```
<Response [200]>
```

```py
resp.raise_for_status()
```

```py
resp.status_code
```

```
200
```

```py
resp.json()["name"]
```

```
'Colin Rundel'
```

```R
api = "https://api.github.com"
resp = request(api) |>
  req_url_path_append("users", "rundel") |>
  req_timeout(10) |>
  req_perform()
resp
```

```
<httr2_response>
GET https://api.github.com/users/rundel
Status: 200 OK
Content-Type: application/json
Body: In memory (1331 bytes)
```

```R
resp_status(resp)
```

```
[1] 200
```

```R
resp_body_json(resp)$name
```

```
[1] "Colin Rundel"
```

Sending the request returns a response. Calling `.json()` or `resp_body_json()` then parses its body. Successful JSON parsing alone does not mean the request succeeded.

A failed request

```py
resp = requests.get(
  api + "/users/not-a-real-user-xyz", timeout=1
)
resp.raise_for_status()
```

```
requests.exceptions.HTTPError: 404 Client
Error: Not Found for url:
https://api.github.com/users/not-a-real-user-
xyz
```

```R
request(api) |>
  req_url_path_append("users", "not-a-real-user
  req_timeout(10) |>
  req_perform()
```

```
Error in `req_perform()`:
! HTTP 404 Not Found.
```

```py
resp.status_code
```

```
404
```

```py
resp.json()["message"]
```

```
'Not Found'
```

```R
resp = last_response()
resp_status(resp)
```

```
[1] 404
```

```R
resp_body_json(resp)$message
```

```
[1] "Not Found"
```

GitHub still sends a response, with a `404` status and a JSON body describing the problem. `httr2` raises an error as soon as it arrives and `requests` only raises if you call `raise_for_status()`, but in both cases the response can still be inspected.

Building a request

A request can be inspected before anything is sent, which is useful for checking that the URL and headers are what we intended.

```py
org_url = api + "/orgs/tidyverse/repos"
req = requests.Request(
  "GET", org_url,
  params={"per_page": 100},
  headers={"User-Agent": "Sta523 demo client"}
).prepare()
```

```R
req = request(api) |>
  req_url_path_append("orgs/tidyverse/repos") |
  req_url_query(per_page = 100) |>
  req_user_agent("Sta523 demo client")
```

```py
req.url
```

```
'https://api.github.com/orgs/tidyverse/repos?
per_page=100'
```

```py
req.headers
```

```
{'User-Agent': 'Sta523 demo client'}
```

```R
req |> req_dry_run()
```

```
GET /orgs/tidyverse/repos HTTP/1.1
accept: */*
accept-encoding: deflate, gzip
host: api.github.com
user-agent: Sta523 demo client
```

`requests.Request` is rarely needed in practice. Passing the same arguments to `requests.get()` builds and sends the

Query parameters

Parameters are given as a dictionary in Python or as named arguments in R. The libraries add them to the URL and handle the encoding.

```py
resp = requests.get(
  org_url,
  params={"per_page": 5, "sort": "updated"},
  timeout=10
)
resp.raise_for_status()
```

```R
resp = request(api) |>
  req_url_path_append("orgs/tidyverse/repos") |
  req_url_query(
    per_page = 5, sort = "updated"
  ) |>
  req_timeout(10) |>
  req_perform()
```

```py
resp.url
```

```
'https://api.github.com/orgs/tidyverse/repos?
per_page=5&sort=updated'
```

```R
resp$url
```

```
[1]
"https://api.github.com/orgs/tidyverse/repos?
per_page=5&sort=updated"
```

```py
[repo["full_name"] for repo in resp.json()]
```

```
['tidyverse/haven',
 'tidyverse/rvest',
 'tidyverse/ggplot2',
 'tidyverse/duckplyr',
 'tidyverse/tidyr']
```

```R
resp_body_json(resp) |>
  purrr::map_chr("full_name")
```

```
[1] "tidyverse/rvest"    "tidyverse/haven"
[3] "tidyverse/ggplot2"  "tidyverse/duckplyr"
[5] "tidyverse/tidyr"
```

Response objects

Content	`requests`	`httr2`
status code	`resp.status_code`, `resp.ok`	`resp_status()`, `resp_is_error()`
status text	`resp.reason`	`resp_status_desc()`
content type	`resp.headers.get("Content-Type")`	`resp_content_type()`
all headers	`resp.headers`	`resp_headers()`
one header	`resp.headers["X-RateLimit-Limit"]`	`resp_header("X-RateLimit-Limit")`
final URL	`resp.url`	`resp$url`

Python's `resp.ok` means status < 400, including redirects, not exactly `200 OK`. Optional headers can be absent, so use

Inspecting a response

```py
resp = requests.get(
  api + "/users/rundel", timeout=10
)
```

```R
resp = request(api) |>
  req_url_path_append("users", "rundel") |>
  req_timeout(10) |>
  req_perform()
```

```py
resp.status_code, resp.reason
```

```
(200, 'OK')
```

```R
resp_status(resp)
```

```
[1] 200
```

```R
resp_status_desc(resp)
```

```
[1] "OK"
```

```py
resp.headers["Content-Type"]
```

```
'application/json; charset=utf-8'
```

```R
resp_content_type(resp)
```

```
[1] "application/json"
```

```py
resp.headers["X-RateLimit-Remaining"]
```

```
'43'
```

```R
resp |> resp_header("X-RateLimit-Remaining")
```

```
[1] "29"
```

Headers carry metadata about the response.

Parsing the body

JSON objects become dictionaries in Python and named lists in R.

py
```
resp.raise_for_status()
user = resp.json()
type(user)
```

```
<class 'dict'>
```

R
```
user = resp_body_json(resp)
class(user)
```

```
[1] "list"
```

py
```
fields = ["login", "name", "public_repos"]
{k: user[k] for k in fields}
```

```
{'login': 'rundel',
 'name': 'Colin Rundel',
 'public_repos': 136}
```

R
```
fields = c("login", "name", "public_repos")
user[fields] |> str()
```

```
List of 3
 $ login       : chr "rundel"
 $ name        : chr "Colin Rundel"
 $ public_repos: int 136
```

Other body types

JSON is the most common response from an API, but the body can be anything. Both libraries expose the raw bytes and the decoded text, and provide parsers for the usual formats.

Content	`requests`	`httr2`
JSON	`resp.json()`	`resp_body_json(resp)`
Text	`resp.text`	`resp_body_string(resp)`
Raw bytes	`resp.content`	`resp_body_raw(resp)`
HTML	`BeautifulSoup(resp.text, "html.parser")`	`resp_body_html(resp)`
XML	`xml.etree.ElementTree.fromstring(resp.content)`	`resp_body_xml(resp)`
CSV	`pd.read_csv(io.StringIO(resp.text))`	`read.csv(text = resp_body_string(resp))`
Save to a file	`open(path, "wb").write(resp.content)`	`req_perform(req, path = path)`

The `Content-Type` header, available via `resp.headers["Content-Type"]` or `resp_content_type(resp)`, says what the

Error handling

The libraries disagree on the default. `httr2` turns a `4xx` or `5xx` status into an R error, while `requests` returns the response and leaves checking to you.

```py
resp = requests.get(api + "/user", timeout=10)
resp.status_code, resp.ok
```

```
(401, False)
```

```py
resp.json()["message"]
```

```
'Requires authentication'
```

```py
resp.raise_for_status()
```

```
requests.exceptions.HTTPError: 401 Client
Error: Unauthorized for url:
https://api.github.com/user
```

```R
request(api) |>
  req_url_path_append("user") |>
  req_timeout(10) |>
  req_perform()
```

```
Error in `req_perform()`:
! HTTP 401 Unauthorized.
```

```R
resp = request(api) |>
  req_url_path_append("user") |>
  req_error(is_error = \(resp) FALSE) |>
  req_timeout(10) |>
  req_perform()
resp_status(resp)
```

```
[1] 401
```

```R
resp_body_json(resp)$message
```

```
[1] "Requires authentication"
```

Pagination

Most APIs cap the number of results in one response, so retrieving a large collection means a sequence of requests. There is no single convention, so check the documentation for each API.

- Page-based, `?page=2&per_page=30`
- Offset-based, `?offset=60&limit=30`
- Cursor-based, each response includes a token that identifies the next batch
- Link header, the response's `Link` header gives the URL of the next page

GitHub uses page-based parameters and also returns a `Link` header, so a client can follow the `next` link until there is none.

GitHub pagination

```py
resp = requests.get(
  org_url,
  params={"per_page": 25}, timeout=10
)
resp.raise_for_status()
len(resp.json())
```

```
25
```

```R
resp = request(api) |>
  req_url_path_append("orgs/tidyverse/repos") |
  req_url_query(per_page = 25) |>
  req_timeout(10) |>
  req_perform()
length(resp_body_json(resp))
```

```
[1] 25
```

```py
resp.links
```

```
{'next': {'url':
'https://api.github.com/organizations/22032646/
repos?per_page=25&page=2',
          'rel': 'next'},
 'last': {'url':
'https://api.github.com/organizations/22032646/
repos?per_page=25&page=2',
          'rel': 'last'}}
```

```R
resp_link_url(resp, "next")
```

```
[1]
"https://api.github.com/organizations/22032646/
repos?per_page=25&page=2"
```

```R
resp_link_url(resp, "last")
```

```
[1]
"https://api.github.com/organizations/22032646/
repos?per_page=25&page=2"
```

`requests` parses the `Link` header into the `links` dictionary, and `httr2` provides `resp_link_url()` to pull out a single relation.

Following links

Collect a page, follow its `next` link, and stop when that link is absent. `httr2` provides an iterator for this pattern.

```py
url = org_url
params = {"per_page": 25}
repos = []

while url is not None:
    resp = requests.get(
        url, params=params, timeout=10
    )
    resp.raise_for_status()
    repos.extend(resp.json())
    next_link = resp.links.get("next", {})
    url = next_link.get("url")
    params = None
```

```R
resps = request(api) |>
  req_url_path_append("orgs/tidyverse/repos") |
  req_url_query(per_page = 25) |>
  req_timeout(10) |>
  req_perform_iterative(
    next_req = iterate_with_link_url(),
    max_reqs = Inf, progress = FALSE
  )

repos = resps |>
  resps_data(resp_body_json)
```

The next URL includes its query parameters, so Python resets `params` after the first request. `max_reqs = Inf` removes R's

Checking the combined results

Authentication and rate limits

Everything so far has been anonymous. Most APIs give anonymous clients a small rate limit and no access to private resources, and require authentication for anything more.

GitHub API	Requests per hour	Access
Unauthenticated	60	Public data only
Authenticated	Usually 5,000	Resources allowed by the token

GitHub accepts a bearer token in the `Authorization` header, sent as `Authorization: Bearer <token>`. Other APIs may use API keys, OAuth, or another scheme.

The token identifies you to the API and grants its assigned permissions. Treat it like a password. GitHub rate limits vary by authentication method, and secondary limits also apply.

Personal access tokens

A personal access token (PAT) is a credential generated on GitHub under Settings > Developer settings > Personal access tokens. Fine-grained tokens scoped to specific repositories and permissions are recommended over classic tokens.

- Never put a token in a script, a notebook, or a git repository.
- Store it outside the code and read it at run time, from an environment variable, a file such as `.env` that is ignored by git, or a credential manager.
- Give it the minimum permissions needed and an expiration date.

```py
import os
token = os.environ["GITHUB_TOKEN"]
```

```R
gitcreds::gitcreds_set()
token = gitcreds::gitcreds_get()$password
```

In R, `gitcreds` stores the token in the operating system's credential store (the same one git uses), so `gitcreds_set()` only needs to be run once per machine. In Python, set the environment variable before running these examples. Never

Authenticated requests

With the token in place, the `/user` endpoint that failed earlier now returns the account the token belongs to.

```py
resp = requests.get(
  api + "/user",
  headers={
    "Authorization": f"Bearer {token}"
  },
  timeout=10
)
resp.raise_for_status()
```

```R
resp = request(api) |>
  req_url_path_append("user") |>
  req_auth_bearer_token(token) |>
  req_timeout(10) |>
  req_perform()
```

```py
resp.status_code
```

```
200
```

```R
resp_status(resp)
```

```
[1] 200
```

```py
resp.json()["login"]
```

```
'rundel'
```

```R
resp_body_json(resp)$login
```

```
[1] "rundel"
```

```py
resp.headers["X-RateLimit-Limit"]
```

```
'5000'
```

```R
resp_header(resp, "X-RateLimit-Limit")
```

```
[1] "5000"
```

`req_auth_bearer_token()` is a convenience for `req_headers(Authorization = paste("Bearer", token))`. Either

Sessions and templates

When making many requests to the same API, set the shared pieces such as the user agent and token once. In `requests` a `Session` carries headers across calls and reuses the connection. In `httr2` a request object is a template that can be extended before each `req_perform()`.

py
```python
s = requests.Session()
s.headers.update({
  "User-Agent": "Sta523 demo client",
  "Authorization": f"Bearer {token}"
})
```

R
```r
gh = request(api) |>
  req_user_agent("Sta523 demo client") |>
  req_auth_bearer_token(token) |>
  req_timeout(10)
```

py
```python
for login in ["hadley", "wesm"]:
    resp = s.get(
        f"{api}/users/{login}", timeout=10
    )
    resp.raise_for_status()
    print(resp.json()["name"])
```

```
Hadley Wickham
Wes McKinney
```

R
```r
for (login in c("hadley", "wesm")) {
  user = gh |>
    req_url_path_append("users", login) |>
    req_perform() |> resp_body_json()
  print(user$name)
}
```

```
[1] "Hadley Wickham"
[1] "Wes McKinney"
```

Sending data

Both libraries serialize a dictionary or list as JSON and set the appropriate `Content-Type`. This example needs a token with permission to write gists (the Gists account permission on a fine-grained token, or the `gist` scope on a classic one), and creates a public gist on the account the token belongs to.

py

```
gist = {
  "description": "Sta 523 demo",
  "files": {"hello.py": {
    "content": "print('hello world')\n"
  }},
  "public": True
}
resp = s.post(
  api + "/gists", json=gist, timeout=10
)
resp.raise_for_status()
```

R

```
gist = list(
  description = "Sta 523 demo",
  files = list("hello.R" = list(
    content = "print('hello world')\n"
  )),
  public = TRUE
)
resp = gh |>
  req_url_path_append("gists") |>
  req_body_json(gist) |> req_perform()
```

Checking a write

Creating a gist returns `201 Created`. Its response body describes the new resource, including its URL and `id`.

```py
resp.status_code
```

```
201
```

```py
resp.reason
```

```
'Created'
```

```py
resp.json()["html_url"]
```

```
'https://gist.github.com/rundel/d1fe67f2391802b4
```

```R
resp_status(resp)
```

```
[1] 201
```

```R
resp_status_desc(resp)
```

```
[1] "Created"
```

```R
resp_body_json(resp)$html_url
```

```
[1] "https://gist.github.com/rundel/aca96b17a5bc
```

A successful write can return a different status and body from a successful read, so check the endpoint documentation.

Cleaning up

A `DELETE` request to `/gists/{gist_id}`, using the `id` returned when the gist was created, removes it. The response has status `204 No Content` and an empty body.

py
```
gist_id = resp.json()["id"]
resp = s.delete(
  f"{api}/gists/{gist_id}", timeout=10
)
resp.raise_for_status()
```

R
```
gist_id = resp_body_json(resp)$id
resp = gh |>
  req_url_path_append("gists", gist_id) |>
  req_method("DELETE") |>
  req_perform()
```

py
```
resp.status_code
```

```
204
```

R
```
resp_status(resp)
```

```
[1] 204
```

py
```
resp.reason
```

```
'No Content'
```

R
```
resp_status_desc(resp)
```

```
[1] "No Content"
```

Rate limiting

A rate limit usually produces a `429`, but some APIs use other `4xx` codes (GitHub sometimes returns a `403`). Check the headers and message before retrying.

```py
import time

users = ["hadley", "wesm", "rundel"]
for user in users:
    try:
        resp = s.get(
            f"{api}/users/{user}", timeout=10
        )
        resp.raise_for_status()
    finally:
        time.sleep(1)
```

```R
gh = gh |>
  req_throttle(
    capacity = 1, fill_time_s = 1
  )

users = c("hadley", "wesm", "rundel")
for (user in users) {
  gh |> req_url_path_append("users", user) |>
    req_perform()
}
```

Both permit one request immediately, then one per second. A capacity of 60 would permit a burst of 60 requests. Choose a rate that fits the API.

Retries

Transient failures of read requests (a timeout, a `503`, a rate limit) are worth retrying after a pause, with the pause growing on each attempt.

`httr2` builds this in with `req_retry()`. The template below allows up to five attempts, waits with exponential backoff, and honors `Retry-After`.

```r
gh = gh |>
  req_retry(
    max_tries = 5, retry_on_failure = TRUE,
    is_transient = \(resp) resp_status(resp) %in% c(429, 500, 502, 503, 504)
  )
```

`requests` has no retry option of its own. Retries are configured on the underlying `urllib3` connection pool by mounting an `HTTPAdapter` with a `Retry` object on a `Session`, which sets the number of attempts, the backoff, and the statuses and methods that qualify. See transport adapters and `urllib3.util.Retry`.

`httr2` does not restrict retries by method, so use this template for reads only. Retrying a write may duplicate it. A GitHub

Exercise 1

Using the GitHub API, retrieve every public repository belonging to the `posit-dev` organization and build a data frame with one row per repository containing its name, primary language, star count, and the date it was last pushed to.

- Request `type=public` and `per_page=100`, and follow every next-page link. How many repositories and pages are there?
- Tabulate the primary language. Treat a missing language as its own category.
- Which five repositories were pushed to most recently?

Write a version in Python with `requests` and a version in R with `httr2`. Authenticate your requests if you have a token available.

Summary

`requests` and `httr2`

Task	`requests`	`httr2`
GET	`requests.get(url)`	`request(url)` then `req_perform()`
timeout	`timeout=10`	`req_timeout(10)`
query parameters	`params={...}`	`req_url_query(...)`
headers	`headers={...}`	`req_headers(...)`
bearer token	`headers={"Authorization": f"Bearer {t}"}`	`req_auth_bearer_token(t)`
send JSON	`requests.post(url, json=...)`	`req_body_json(...)`
other methods	`requests.put()`, `.patch()`, `.delete()`	`req_method()`
inspect before sending	`requests.Request(...).prepare()`	`req_dry_run()`
status	`resp.status_code`, `resp.ok`	`resp_status()`, `resp_is_error()`
raise on error	`resp.raise_for_status()`	default, disable with `req_error()`
parse JSON	`resp.json()`	`resp_body_json()`
headers	`resp.headers`	`resp_headers()`, `resp_header()`
pagination	`resp.links["next"]["url"]` in a loop	`req_perform_iterative()`, `resps_data()`
shared settings	`requests.Session()`	a request object used as a template
throttle / retry	`time.sleep()`, `urllib3.util.Retry`	`req_throttle()`, `req_retry()`

Takeaways

- HTTP is a request with a method, URL, headers, and body, answered by a response with a status, headers, and body. Everything on the web, including scraping and APIs, is built on this exchange.
- A REST API exposes resources at URLs using HTTP methods and representations such as JSON. The status code says whether it worked and the JSON body carries the result.
- `requests` does each request in a single function call, while `httr2` builds a request object and then performs it. Both give back a response object with the status, headers, and methods for parsing the body.
- Check the status. `httr2` errors by default and `requests` does not, so call `raise_for_status()` before parsing. Set a timeout as well.
- Large collections come back a page at a time. Follow the `Link` header or page parameters until the results run out.
- Authenticate with a token kept out of your code, respect rate limits, and be careful with any request that is not a `GET`.